LMA PROJECT PROPOSAL

Team - SSSS

Team Members:

Akash Sigullapalli - 2023101104

Konapalli Jaswanth Reddy - 2023101025

Kotha Sai Veekshith - 2023101078

Srinivas Billuri - 2023101029

Project Title

Intent-Gated Data Transformation Agent

Domain and Target User

Domain: Data engineering and analytics

Target users: Data analysts, data engineers, and developers who use LLMs to generate or automate data transformations.

Problem Statement

LLMs are increasingly being used to generate Python code for common data transformations such as filtering, deduplication, missing-value handling, type conversion, and column transformation. Although the generated code may execute successfully, it can still make changes that were not requested by the user. For example, when a user asks to remove duplicate customers, an LLM may accidentally remove valid records or modify unrelated columns while producing code that runs without any error. Existing data-quality approaches commonly check properties such as schema validity, missing values, uniqueness, and value ranges, while recent LLM-based data-cleaning systems also use profiling, execution-based checks, and repair. However, these checks do not directly answer whether the **actual changes made by a transformation match what the user intended to change and whether unrelated data was preserved**. Our project addresses this by representing each transformation request as an explicit intent contract describing both the properties that should change and the properties that must remain unchanged. After executing the generated transformation, the system compares the before-and-after datasets against this contract and detects semantic violations even when the generated code executes successfully. If a violation is detected, the system can attempt a targeted repair and verify the result again.

Research Question

Does an intent contract that specifies what must change and what must be preserved improve the detection of silent semantic errors in LLM-generated data transformations compared with conventional data-quality checks?

Hypothesis

An intent-based verification mechanism will reduce the number of incorrect transformations that are silently accepted, while maintaining a low false-rejection rate for correct transformations.

Why the Problem Matters in This Domain

Incorrect transformations can silently corrupt datasets even when the generated code runs without errors. Such errors can propagate into dashboards, statistical analysis, machine-learning pipelines, and business decisions. A reliable transformation agent therefore needs to verify not only whether the output is technically valid, but whether it performs the **specific transformation requested by the user**.

What Is Missing in Existing Approaches

Traditional data-quality systems provide explicit checks for properties such as uniqueness, null values, schema, distributions, and cross-column integrity. Recent LLM-based data-cleaning agents add profiling, execution, and conservative repair, so simply adding an invariant checker would not be sufficiently novel. Our focus is narrower: **intent compliance of the transformation itself**. We explicitly represent both **what should change** and **what should be preserved**, then measure whether the agent introduces unintended changes even when ordinary data-quality checks pass.

Three Capabilities Used

1. Fine-tuning

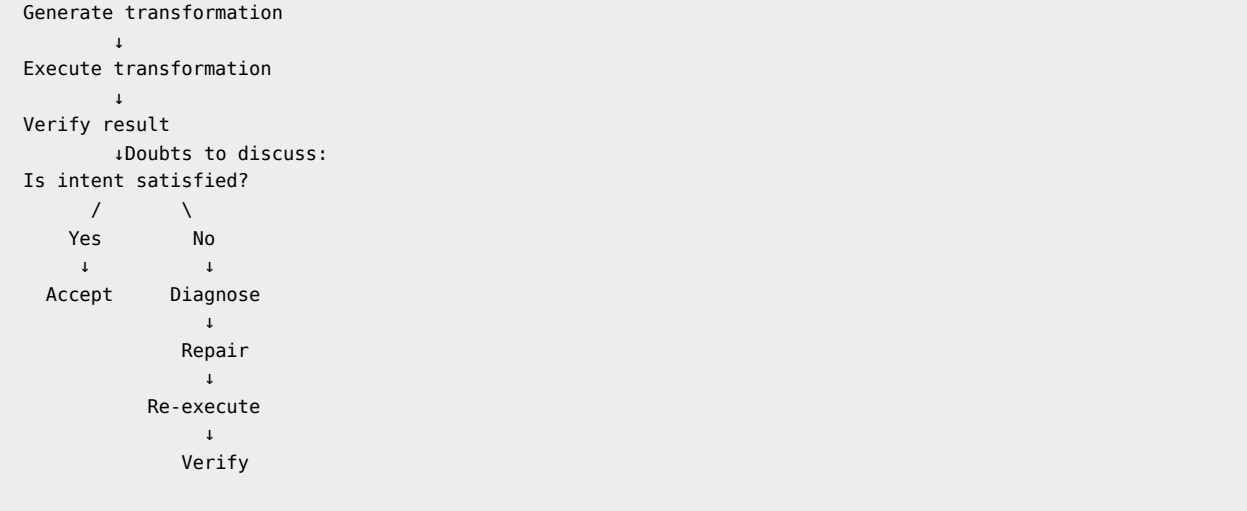
A small open model will be fine-tuned with LoRA/QLoRA to convert natural-language transformation requests into a structured intent contract.

Why needed: The system needs consistent structured representations of transformation intent. Fine-tuning allows the parser to learn the fixed contract format and predicate vocabulary used by the verifier.

2. Agentic Capability (Bounded Repair Loop)

The system will operate as a bounded verification-and-repair loop:

```
Understand request
        ↓
Create intent contract
        ↓
```



The repair process will be limited to a maximum of **2–3 attempts**.

Why needed: Detecting an error is useful, but the system should also attempt to correct the generated transformation instead of only rejecting it.

3. Tool Calling

The system will use tools to:

- execute Python/Pandas transformations
- inspect datasets
- compare before/after datasets
- verify intent contracts
- run transformations inside a sandbox

Why needed:

The system must verify transformations using the actual dataset rather than relying only on the LLM's textual reasoning.

Proposed Idea and Architecture



```
              Generate Python/Pandas
                      ↓
                Safety Checker
                      ↓
                 Sandbox Execute
                      ↓
              Before/After Profiler
                      ↓
              Intent Verification
                /        |        \
           PASS     VIOLATION   UNCERTAIN
             ↓          ↓          ↓
         Accept      Repair      Flag/Ask
                       ↓
                   Re-execute
                       ↓
                    Verify
```

Intent Contract

The intent contract will use a **small closed vocabulary of machine-checkable predicates** rather than arbitrary free-text descriptions.

The core predicate vocabulary will include:

- `row_count`
- `rows_removed_where`
- `duplicate_count`
- `null_count`
- `column_values_unchanged`
- `columns_unchanged`
- `column_dtype`

A gold intent contract is a trusted, human-prepared description of what the system is supposed to change and what it must preserve for a particular user request.

Think of it as the answer key for your fine-tuned model.

- `column_value_relation`

Each predicate will have a deterministic verifier implementation. Conditions will use a normalized structured format rather than free-form strings.

Example:

```
{
  "must_change": [
    {
      "type": "rows_removed_where",
      "target": "price",
      "condition": {
```

```
        "column": "price",
        "operator": "<",
        "value": 0
      }
    }
  ],
  "must_preserve": [
    {
      "type": "column_values_unchanged",
      "target": "customer_id"
    },
    {
      "type": "columns_unchanged",
      "target": ["name", "host_id"]
    }
  ]
}
```

Contract matching will be deterministic and based on normalized fields such as:

predicate type + target + normalized condition/value

This allows the intent parser to be evaluated using standard Precision, Recall, and F1 without using an LLM judge.

Expected Novelty

The novelty of the project is **not** simply using LLMs for data cleaning, using invariants, or adding execution-based verification. These approaches already exist in related data-quality and agentic data-cleaning systems.

The proposed contribution is the use of a **bidirectional, intent-conditioned transformation contract**.

```
What the transformation is expected to change
                +
What the transformation is required to preserve
```

The evaluation will specifically test **silent semantic errors that can pass ordinary data-quality checks**. The project will compare intent-based verification against conventional invariant checking and no verification, rather than claiming that data-quality validation or agentic data cleaning itself is new.

Dataset / Evaluation Environment

The evaluation will use **real publicly available datasets**, not a fully synthetic dataset.

Primary datasets

- **NYC Airbnb Open Data** — real-world listing data with numerical, categorical, geographic, and missing-value fields.

- **UCI Adult** — 48,842 real records with categorical and numerical attributes and missing values.
- **UCI Wine Quality** — 4,898 real records with numerical attributes and quality labels.

Primary Evaluation Tasks

We will manually specify approximately **60–80 transformation tasks** over these real datasets.

The tasks will be restricted to five transformation types:

- filtering
- deduplication
- missing-value handling
- type/format conversion
- column transformation

The task requests will be **hand-written or paraphrased** and kept separate from the automatically constructed examples used for fine-tuning.

Secondary External-Validity Evaluation

We will additionally evaluate on approximately **20–30 selected DS-1000 Pandas problems**.

Only problems with clear and measurable **must-change and must-preserve semantics** will be selected. The DS-1000 results will be reported separately from the real-dataset task results.

Ground Truth

For each evaluation task, we will prepare the correct reference transformation and a **gold intent contract** specifying what must change and what must remain unchanged. We will compare the model-generated contract against this gold contract to measure its accuracy.

Baselines

Baseline 1 — Direct LLM

The LLM directly generates and executes the transformation without any verification.

```
Request → LLM → Transformation → Output
```

This tells us how well an LLM performs without any verification mechanism.

Baseline 2 — Conventional Invariant Checking

```
Request → LLM → Transformation → Conventional data-quality checks → Accept / Reject
```

The fixed checks can include:

- Schema validity
- Null values
- Data types
- Uniqueness
- Value ranges
- Row/column consistency
- Distribution changes

This baseline does **not** use the structured intent contract.

This tells us whether intent-based verification provides benefits beyond standard data-quality checks.

Baseline 3 — Intent Verification with General LLM

The same downstream architecture as the proposed system is used, but the intent parser is a general-purpose LLM instead of the fine-tuned LLM.

```
Request → General-Purpose LLM → Intent Contract → Transformation → Intent Verification → Accept / Reject
```

This tells us whether fine-tuning the intent parser provides an additional benefit.

Proposed System

```
Request → Fine-Tuned LLM → Intent Contract → Transformation → Intent Verification → Accept / Reject
```

The main Stage 1 comparison is therefore:

System	Verification	Intent Parser
Baseline 1	None	—
Baseline 2	Conventional data-quality checks	—
Baseline 3	Intent-based verification	General LLM
Proposed	Intent-based verification	Fine-tuned LLM

Stage 1 — Verification Only

In the first stage, all systems are evaluated in **accept/reject mode only**.

No repair is allowed.

This gives a clean comparison of:

1. no verification,
2. conventional verification,
3. intent-based verification with a general parser,
4. intent-based verification with the fine-tuned parser.

The primary Stage 1 metrics are:

- silent semantic error rate
- intent-violation recall
- false-rejection rate

Stage 2 — Repair Evaluation

The second stage enables the same **bounded repair loop** for the verification-based systems.

The repair process is capped at **2–3 attempts** for every system using repair.

This stage measures:

- repair success
- final transformation accuracy
- average repair attempts

Baseline 1 remains the no-verification reference because it has no violation detector from which to trigger an intent-based repair.

Oracle-Contract Ablation

An additional ablation will provide the verifier with the **gold contract** instead of the parser-generated contract.

```
Gold Contract → Transformation → Same Intent Verifier
```

This keeps the verifier unchanged and isolates the effect of contract-generation errors.

The performance gap between the predicted-contract and gold-contract settings estimates how much of the overall error comes from **misunderstanding user intent during contract generation**.

Metrics

Contract Quality Metrics

Must-change and must-preserve requirements will be evaluated separately.

- **Must-Change Precision**
- **Must-Change Recall**
- **Must-Change F1**
- **Must-Preserve Precision**
- **Must-Preserve Recall**
- **Must-Preserve F1**
- overall structured contract accuracy

No LLM judge will be used. Contract comparison will use the normalized closed predicate vocabulary.

Primary Verification Metrics

- **Silent semantic error rate:** incorrect transformations accepted by the system.
- **Intent-violation recall:** percentage of incorrect transformations detected.
- **False rejection rate:** percentage of correct transformations incorrectly rejected.

Repair Metrics

- **Repair success rate:** percentage of detected errors successfully repaired.
- **Final transformation accuracy:** percentage of tasks producing the correct final dataset.
- **Average repair attempts:** average number of repair iterations used.
- **Improvement from repair:** change in correctness before and after enabling the repair loop.

Capability-Specific Metrics

Fine-tuning:

Must-change and must-preserve Precision/Recall/F1 and overall contract accuracy.

Tool calling:

Valid tool-call rate, correct execution rate, and successful verification execution.

Agentic capabilities (Bounded repair loop):

Repair success rate, final accuracy, average number of attempts, and unnecessary retries.

Safety Metrics

- unsafe Python execution rate
- unauthorized file/network access rate
- unauthorized tool-call rate

Target:

0 unsafe or unauthorized actions.

All code execution will occur inside a restricted sandbox with appropriate filesystem/network restrictions and execution limits.

Compute Plan

- **Base model:** small open model such as Qwen2.5-1.5B/3B or a similar small open model.
- **Fine-tuning:** LoRA/QLoRA on approximately **3,000–5,000 automatically constructed intent examples**.
- **Training compute:** approximately **2–4 GPU hours** on a single 16–24 GB GPU.
- **Inference:** primarily local inference using open models.

The project will prioritize small/open models and local inference to keep the compute and API requirements low.

Expected Risks and Fallback Plan

Risk 1: Intent contracts may be too difficult to generate reliably.

Fallback: restrict the project to the fixed set of transformation types and closed contract predicates.

Risk 2: Some transformations may not have clear preservation properties.

Fallback: remove ambiguous tasks and focus on transformations with measurable before/after effects.

Risk 3: Gold intent contracts may contain annotation ambiguity.

Fallback: have a second teammate independently annotate a sample, measure agreement, refine ambiguous contract definitions before final evaluation, and exclude tasks that remain ambiguous.

Risk 4: The intent-based method may not outperform generic invariants on all tasks.

Fallback: analyze which classes of silent semantic errors are specifically detected by intent verification, even if overall improvement is limited.